

外部任务行为与姿态数据接口说明（Web 对接版）

适用于 PLATFORM_CONTROL 与 WEAPON_FIRING 外部任务

协议版本：1.0
传输方式：WebSocket
发送策略：状态变化触发：开始 *_start，结束 *_end
任务关联：服务端解析 task_runs.task_id
时间基准：Unix 毫秒时间戳
修订日期：2026-07-29

1. 文档目的

本文档定义“平台控制”和“武器发射”两类外部任务的行为数据、姿态数据及其任务关联规则，用于 UE 外部仿真平台与 F18-Radar Web 服务端联调、数据落库和后续实验分析。

适用任务类型：

任务	task_type
平台控制	PLATFORM_CONTROL
武器发射	WEAPON_FIRING

本文档约定：

- 仅在操纵杆或按钮状态发生变化时生成行为事件。
- 操作开始发送 *_start，松开或回中发送 *_end；持续保持期间不得重复发送。
- 行为事件记录操作者、任务类型、按键、行为含义和发生时间。
- 行为事件同时携带操作发生时的当前姿态快照。

- `pose_data` 的实体数量应与当前任务配置中的 `units` 数量一致。
- 外部客户端可以不提供 `task_id`，由服务端根据当前活动任务解析。
- 所有客户端业务时间戳统一使用 Unix 毫秒时间戳。

2. 通信方式

行为和姿态数据通过现有 WebSocket 连接发送。

为避免行为字段 `type` 与 WebSocket 消息类型冲突，消息采用“外层消息类型 + 内层业务数据”的结构：

```
JSON
{
  "type": "external_behavior_record",
  "data": {
    "type": "y_axis_start"
  }
}
```

其中：

- 外层 `type` 表示 WebSocket 消息类型。
- `data.type` 表示本次行为的业务类型。

3. 通用约定

3.1 字段命名

外部协议已有的 `PlayerID` 字段继续保留，服务端接收后统一映射为内部 `user_id`。

原始草案中的 "`pose data`" 统一规范为合法、稳定的 JSON 字段名：

```
TEXT
pose_data
```

3.2 时间单位

客户端发送的以下字段均使用 Unix 毫秒时间戳：

- 行为事件 `timestamp`
- 姿态记录 `timestamp`

- 姿态快照 `captured_at`

示例：

```
JSON
{
  "timestamp": 1720000000000
}
```

服务端可以另外记录微秒级接收时间，但不得覆盖客户端原始时间。

3.3 任务类型

接口仅接受以下大写枚举值：

```
TEXT
PLATFORM_CONTROL
WEAPON_FIRING
```

现有代码中的 `platform_control`、`weapon_launch` 仅作为内部路由分类，不作为本接口对外值。

3.4 task_id 规则

`task_id` 为可选字段。

外部客户端可以发送：

```
JSON
{
  "task_id": null
}
```

当 `task_id` 缺失或为 `null` 时，服务端必须使用以下条件解析当前具体子任务：

```
TEXT
user_id = PlayerID
task_type = PLATFORM_CONTROL 或 WEAPON_FIRING
status = active
```

解析顺序：

1. 优先使用外部任务内存上下文中的 `gaze_task_id` 或 `current_subtask_task_id`。
2. 内存上下文不存在时，查询 `task_runs` 中相同用户、相同任务类型的活动记录。
3. 只找到一条记录时，使用该记录的 `task_id`。
4. 找到多条且无法通过当前上下文消歧时，拒绝写入。
5. 未找到活动任务时，拒绝写入。

数据库最终保存的必须是具体子任务 `task_runs.task_id`，不得使用 `task_groups.group_id` 代替。

4. 行为数据记录接口

4.1 接口语义

行为数据记录表示：

某个用户或 AI 在某个具体任务下，于某一时刻执行了一次按键或控制操作，并同时保存该时刻的当前姿态数据。

发送端只在状态边沿生成记录：

- 操作开始：发送一次对应的 *_start。
- 操作结束、按钮松开或轴回中：发送一次对应的 *_end。
- 操作持续保持：不得在轮询或 Tick 中重复发送相同状态。

4.2 完整请求示例

```
JSON
{
  "type": "external_behavior_record",
  "schema_version": "1.0",
  "client_event_id": "7d72c9dc-2674-483e-bf10-c77035379a56",
  "data": {
    "PlayerID": "P001",
    "task_type": "PLATFORM_CONTROL",
    "task_id": null,
    "button": "button4",
    "type": "y_axis_start",
    "timestamp": 1785123456789,
    "owner": "User",
    "pose_data": [
      {
        "Side": "Our",
        "ID": "101",
        "timestamp": 1785123456789,
        "posture": {
          "Pitch": 5.2,
          "Yaw": 180.0,
          "Roll": -12.4
        }
      },
      {
        "Side": "Enemy",
        "ID": "201",
        "timestamp": 1785123456789,
        "posture": {
          "Pitch": 1.8,
          "Yaw": 92.5,
          "Roll": 3.0
        }
      }
    ]
  }
}
```

4.3 顶层字段

字段	类型	必填	说明
type	string	是	固定为 external_behavior_record
schema_version	string	是	当前固定为 1.0
client_event_id	string/UUID	是	客户端事件唯一 ID，用于幂等写入
data	object	是	行为事件业务数据

4.4 data 字段

字段	类型	必填	说明
PlayerID	string	是	当前用户/被试 ID，服务端映射为 <code>user_id</code>
task_type	string	是	PLATFORM_CONTROL 或 WEAPON_FIRING
task_id	string/integer/ null	否	具体子任务 ID；为空时由服务端解析
button	string/null	条件必填	当前三轴映射使用 <code>button2 ~ button7</code> ；无物理按钮来源时可为 <code>null</code>
type	string	是	行为类型，例如 <code>x_axis_start</code> 、 <code>y_axis_end</code>
timestamp	integer	是	行为发生时间，Unix 毫秒
owner	string	是	本次行为的执行方：User 或 AI
pose_data	array	是	行为发生时的当前姿态快照；数量应与当前任务配置的 <code>units</code> 数量一致

4.5 三轴与物理按钮映射

轴/方向	Value	button	data.type
X 左	-1	button2	<code>x_axis_start / x_axis_end</code>
X 右	1	button3	<code>x_axis_start / x_axis_end</code>
Y 上	1	button4	<code>y_axis_start / y_axis_end</code>
Y 下	-1	button5	<code>y_axis_start / y_axis_end</code>
Z 左	-1	button6	<code>z_axis_start / z_axis_end</code>
Z 右	1	button7	<code>z_axis_start / z_axis_end</code>

`button` 用于识别具体物理方向，`data.type` 用于判断行为类别和开始/结束状态。`button1` 当前不在三轴映射中；如需用于开火或其他独立按钮，应在双方确认映射后再启用。

4.6 行为类型枚举

枚举值	中文含义
x_axis_start	X 轴方向操作开始
x_axis_end	X 轴方向操作结束或回中
y_axis_start	Y 轴方向操作开始
y_axis_end	Y 轴方向操作结束或回中
z_axis_start	Z 轴方向操作开始
z_axis_end	Z 轴方向操作结束或回中

当前 Web 对接版按 `data.type` 判断行为类别，按 `data.button` 判断具体物理方向。若后续需要记录开火、切换模式、确认按钮或连续轴值，应在协议中新增明确枚举或独立的 `value` 字段，不得把数值拼接进 `button` 或 `type`。

4.7 发送状态规则

当前状态变化	发送行为
未操作 → 开始操作	发送对应轴的 *_start
保持操作	不发送
操作 → 松开/回中	发送对应轴的 *_end
持续松开/回中	不发送

同一物理方向必须先有 *_start，再有对应的 *_end。重复的连续 *_start 或连续 *_end 应由发送端抑制，服务端也应结合 `client_event_id` 做幂等保护。

4.8 执行方枚举

枚举值	中文含义	内部兼容值
User	用户执行	manual
AI	AI 执行	AI

对外协议必须保留 User/AI；如复用现有 `user_operations.event_owner`，服务端负责将 User 转换为 manual。

4.9 AI 行为示例

```
JSON
{
  "type": "external_behavior_record",
  "schema_version": "1.0",
  "client_event_id": "13907979-eed5-413c-8910-d2db4248c18a",
  "data": {
    "PlayerID": "P001",
    "task_type": "WEAPON_FIRING",
    "task_id": null,
    "button": "button7",
    "type": "z_axis_end",
    "timestamp": 1785123459000,
    "owner": "AI",
    "pose_data": [
      {
        "Side": "Our",
        "ID": "101",
        "timestamp": 1785123459000,
        "posture": {
          "Pitch": 4.7,
          "Yaw": 181.2,
          "Roll": -8.5
        }
      },
      {
        "Side": "Enemy",
        "ID": "201",
        "timestamp": 1785123459000,
        "posture": {
          "Pitch": 0.5,
          "Yaw": 95.0,
          "Roll": 2.6
        }
      }
    ]
  }
}
```

5. 姿态数据结构

5.1 接口语义

姿态数据表示某个实体在某一时刻的当前姿态角。

行为事件中的 `pose_data` 是操作发生时的快照，不是指向后续可变状态的引用。行为记录写入后，对应姿态快照不得被后续位置更新覆盖。

5.2 姿态对象

```
JSON
{
  "Side": "Our",
  "ID": "101",
  "timestamp": 1785123456789,
  "posture": {
    "Pitch": 5.2,
    "Yaw": 180.0,
    "Roll": -12.4
  }
}
```

5.3 姿态字段

字段	类型	必填	说明
Side	string	是	实体所属阵营: Our 或 Enemy; UE 内部 side id=1 映射为 Our, side id=2 映射为 Enemy
ID	string	是	实体唯一 ID, 对应当前任务配置中的 units.id
timestamp	integer	是	本条姿态的采集时间, Unix 毫秒
posture	object	是	当前姿态角对象
posture.Pitch	number	是	俯仰角
posture.Yaw	number	是	航向角/偏航角
posture.Roll	number	是	横滚角

posture 必须使用结构化数值对象, 不得使用 "Pitch,Yaw,Roll" 拼接字符串。行为消息中的 pose_data 数量必须与当前任务配置中的 units 数量一致, 每个 units.id 应有且仅有一条对应姿态。

5.4 阵营枚举

枚举值	中文含义
Our	我方
Enemy	敌方

同一姿态快照中允许存在多个我方或敌方实体, 每个实体使用独立 ID。

5.5 姿态角单位

外部仿真平台必须保证同一任务中的 `Pitch`、`Yaw`、`Roll` 使用相同角度单位。当前示例按角度值表达，联调时应明确使用度或弧度；在单位正式确认前，服务端只负责原值保存，不做换算。

如需显式声明单位，建议在姿态快照中增加：

```
JSON
{
  "angle_unit": "degree"
}
```

6. 独立姿态快照接口

行为记录已经携带当前姿态数据。若外部平台还需要在没有行为事件时独立记录姿态变化，可以使用本接口。

6.1 请求示例

```
JSON
{
  "type": "external_pose_record",
  "schema_version": "1.0",
  "client_snapshot_id": "366dce79-7610-4d3d-b335-e6d126fe27a4",
  "data": [
    {
      "PlayerID": "P001",
      "task_type": "PLATFORM_CONTROL",
      "task_id": null,
      "captured_at": 1785123456789,
      "pose_data": [
        {
          "Side": "Our",
          "ID": "101",
          "timestamp": 1785123456789,
          "posture": {
            "Pitch": 5.2,
            "Yaw": 180.0,
            "Roll": -12.4
          }
        },
        {
          "Side": "Enemy",
          "ID": "201",
          "timestamp": 1785123456789,
          "posture": {
            "Pitch": 1.8,
            "Yaw": 92.5,
            "Roll": 3.0
          }
        }
      ]
    }
  ]
}
```

6.2 字段

字段	类型	必填	说明
type	string	是	固定为 external_pose_record
schema_version	string	是	当前固定为 1.0
client_snapshot_id	string/UUID	是	客户端姿态快照唯一 ID
data.PlayerID	string	是	当前用户/被试 ID
data.task_type	string	是	PLATFORM_CONTROL 或 WEAPON_FIRING
data.task_id	string/integer/ null	否	具体子任务 ID；为空时由服务端解析
data.captured_at	integer	是	整组姿态快照采集时间，Unix 毫秒
data.pose_data	array	是	当前姿态对象数组；数量应与当前任务配置的 units 数量一致

独立姿态接口用于低频状态快照。若需要持续高频记录原始仿真姿态流，应另行定义采样频率、批量格式、压缩和文件存储方案，避免高频逐条写入主 SQLite 数据库。

7. 行为与姿态关联规则

7.1 原子记录

推荐流程：

TEXT

行为发生

- 生成 client_event_id
- 读取当前姿态
- 形成 pose_data 快照
- 一次发送 external_behavior_record
- 服务端在同一事务中写入行为和姿态

行为数据和姿态数据必须关联到同一个解析后的 task_id。

7.2 时间关系

正常情况下：

TEXT

行为 timestamp ≈ 姿态 timestamp

如果外部平台不能同时采集，应选择行为发生时刻之前最近的一份有效姿态，不得使用行为发生后的未来姿态冒充当前姿态。

7.3 幂等规则

- `client_event_id` 在行为事件中必须唯一。
- `client_snapshot_id` 在独立姿态快照中必须唯一。
- 服务端收到重复 ID 时返回原写入结果，不得重复插入。

8. 成功响应

8.1 行为记录成功

JSON

```
{
  "type": "external_behavior_record_result",
  "ok": true,
  "client_event_id": "7d72c9dc-2674-483e-bf10-c77035379a56",
  "behavior_record_id": 1001,
  "resolved_task_id": 42,
  "task_type": "PLATFORM_CONTROL",
  "pose_record_count": 2,
  "duplicate": false,
  "server_time_us": 1720000000123456
}
```

8.2 姿态记录成功

JSON

```
{
  "type": "external_pose_record_result",
  "ok": true,
  "client_snapshot_id": "366dce79-7610-4d3d-b335-e6d126fe27a4",
  "pose_snapshot_id": 2001,
  "resolved_task_id": 42,
  "task_type": "PLATFORM_CONTROL",
  "pose_record_count": 2,
  "duplicate": false,
  "server_time_us": 1720000000123456
}
```

9. 错误响应

```
JSON
{
  "type": "external_behavior_record_result",
  "ok": false,
  "client_event_id": "7d72c9dc-2674-483e-bf10-c77035379a56",
  "error": {
    "code": "ACTIVE_TASK_NOT_FOUND",
    "message": "未找到当前用户和任务类型对应的活动子任务"
  }
}
```

建议错误码：

错误码	含义
INVALID_PAYLOAD	JSON 结构或字段类型错误
UNSUPPORTED_TASK_TYPE	不支持的任务类型
UNSUPPORTED_BEHAVIOR_TYPE	不支持的行为类型
INVALID_BUTTON	按钮编号不合法
INVALID_OWNER	owner 不是 User 或 AI
INVALID_TIMESTAMP	时间戳缺失或不是毫秒整数
INVALID_POSE_DATA	姿态结构、实体数量或姿态角数值不合法
TASK_ID_MISMATCH	客户端 task_id 与当前活动任务不一致
ACTIVE_TASK_NOT_FOUND	未找到匹配的活动子任务
ACTIVE_TASK_AMBIGUOUS	找到多条活动子任务且无法消歧
INTERNAL_ERROR	服务端写入失败

10. 数据库存储建议

表职责约定：

- task_runs 只保存一次具体子任务的生命周期和任务上下文，是行为、姿态记录的关联目标。
- task_groups 只保存任务组汇总和进度，不承载行为或姿态明细。
- task_settings 是任务配置快照，不适合保存高频事件或姿态数据。

- 外部请求中的 `task_id` 可以为 `null`，但服务端解析成功后，正式业务记录必须保存具体的 `task_runs.task_id`。

10.1 行为数据

行为事件可以复用现有 `user_operations`：

接口字段	<code>user_operations</code>
服务端解析出的 <code>task_id</code>	<code>task_id</code>
<code>data.type</code>	<code>operation_type</code>
<code>data.timestamp</code>	<code>timestamp</code>
服务端接收时间	<code>receive_timestamp</code>
PlayerID	<code>user_id</code>
<code>owner=User</code>	<code>event_owner>manual</code>
<code>owner=AI</code>	<code>event_owner=AI</code>
<code>button</code> 、原始 <code>owner</code> 、 <code>client_event_id</code> 、原始 <code>pose_data</code> 等	<code>parameters JSON</code>

`is_active` 建议按状态类型映射：`*_start` 写入 1，`*_end` 写入 0。

为保证 `client_event_id` 幂等，如果复用 `user_operations`，应新增可唯一索引的客户端事件 ID 字段，或新增独立幂等键表。仅将 `client_event_id` 放入 `parameters JSON` 不能由数据库唯一约束阻止重复写入。

如果需要对外部行为做独立统计，也可以新增 `external_behavior_records`，但不得同时把同一事件重复计入两套业务统计。

10.2 姿态数据

建议新增独立姿态表 `external_pose_records`，每个实体一行：

字段	说明
<code>id</code>	自增主键
<code>pose_snapshot_id</code>	姿态快照 ID
<code>behavior_operation_id</code>	关联 <code>user_operations.id</code> ，独立快照时可为空
<code>client_snapshot_id</code>	独立姿态快照客户端 ID
<code>task_id</code>	关联 <code>task_runs.task_id</code>
<code>user_id</code>	用户 ID
<code>task_type</code>	PLATFORM_CONTROL 或 WEAPON_FIRING
<code>side</code>	Our 或 Enemy
<code>entity_id</code>	实体 ID
<code>pose_time_ms</code>	姿态采集时间
<code>pitch、yaw、roll</code>	当前姿态角
<code>angle_unit</code>	角度单位，可为空；建议明确为 <code>degree</code> 或 <code>radian</code>
<code>raw_payload_json</code>	原始姿态对象
<code>created_at</code>	数据库写入时间

建议索引：

TEXT

(task_id, pose_time_ms)

(task_id, side, entity_id, pose_time_ms)

(behavior_operation_id)

10.3 事务要求

处理 `external_behavior_record` 时，行为事件和其 `pose_data` 必须在同一数据库事务中写入：

TEXT

行为写入成功 + 所有姿态写入成功 → 提交

任一写入失败 → 全部回滚

不得出现行为记录已保存、对应姿态只保存一部分的状态。

10.4 当前落地前置项

截至本文档修订时，正式接入前还需要完成：

6. 新增 `external_behavior_record` / `external_pose_record` 的 WebSocket 处理器。
7. 将活动任务回退查询改为“唯一候选才接受”，不得在多条活动记录中直接选择最新一条。
8. 为 `client_event_id` 和 `client_snapshot_id` 建立数据库级幂等约束。
9. 创建 `external_pose_records` 表，并将行为及其全部姿态放入同一事务。

11. 联调检查清单

- ☐ `task_type` 使用 `PLATFORM_CONTROL` 或 `WEAPON_FIRING`
- ☐ `PlayerID` 与外部任务启动消息中的 ID 一致
- ☐ `task_id` 缺失时能从当前活动 `task_runs` 解析
- ☐ 操作开始发送一次 `*_start`
- ☐ 松开或回中发送一次 `*_end`
- ☐ 持续保持期间不重复发送
- ☐ `timestamp` 为 Unix 毫秒
- ☐ `owner` 只使用 `User` 或 `AI`
- ☐ X/Y/Z 三轴分别按 `button2 ~ button7` 映射
- ☐ `data.type` 使用对应轴的 `*_start` 或 `*_end`
- ☐ `pose_data` 是数组
- ☐ `pose_data` 数量与当前任务配置中的 `units` 数量一致
- ☐ `pose_data[].ID` 对应 `units.id`
- ☐ `posture` 使用 `Pitch`、`Yaw`、`Roll` 数值对象，不使用字符串
- ☐ 行为与姿态写入同一个具体子任务
- ☐ 重复 `client_event_id` 不产生重复数据
- ☐ 成功响应返回 `resolved_task_id`